

Reviewer Pack — Minimal Order of Modular Forms and Resolution Proof Length I...

Entry 16fe2475ef54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 23:03:20 UTC

Minimal Order of Modular Forms and Resolution Proof Length Inequality

Entry ID: 16fe2475ef54 Verdict: INCONCLUSIVE Recorded: 2026-06-05 23:03:20 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Modular Forms) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G associated with a d -regular graph G , the minimal order of modular forms for φ_G is linearly correlated with its resolution proof length $L(\varphi_G)$, such that the minimal order of modular forms = $\Theta(L(\varphi_G))$.

Rationale (proposer’s reasoning):

Modular forms are highly structured objects in number theory that may encode information about the complexity of computational problems. Their study could reveal new insights into the structure of resolution proofs.

Taxonomy category: MODULAR_FORMS_RESOLUTION_PROOF (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a162bf5d6d40bc6e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Tseitin formula φ_G , if the minimal order of modular forms equals $\Theta(L(\varphi_G))$ with a correlation coefficient ≥ 0.7 and $p\text{-value} \leq 0.05$ over 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal order modular forms AND resolution proof complexity - Tseitin formula d -regular graph AND modular form order - resolution proof length inequality AND $\emptyset$ (minimal order modular forms)

Top relevant hits considered: - [http://arxiv.org/abs/2004.05156v2] Generating series of all modular graph forms from iterated Eisenstein integrals - [http://arxiv.org/abs/1908.05124v2] Minimal Representations of Order Types by Geometric Graphs - [http://arxiv.org/abs/0804.4681v1] d -Regular graphs of acyclic chromatic index at least $d + 2$ - [http://arxiv.org/abs/2105.12054v4] A proof of Gromov’s cube inequality on scalar curvature - [http://arxiv.org/abs/1708.09775v13] Resolution of singularities and geometric proofs of the Łojasiewicz inequalities - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly’s Eye Exp

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n, d):
        if n % d != 0:
            return None
        variables = list(range(1, n + 1))
        clauses = []
        for i in range(1, n + 1):
            clause = [random.choice([-1, 1]) * v for v in variables if v != i]
            clauses.append(clause)
        return clauses

    def resolution_length(clauses):
        stack = list(clauses)
        while True:
            new_clause = None
            for i in range(len(stack)):
                for j in range(i + 1, len(stack)):
                    clause_i = set(abs(lit) for lit in stack[i])
                    clause_j = set(abs(lit) for lit in stack[j])
                    if len(clause_i & clause_j) == 1:
                        new_clause = [lit for lit in stack[i] if abs(lit) not in clause_j]
                        break
            if new_clause is not None:
                stack.remove(new_clause)
            else:
                break
        return len(stack)
```

```

        if new_clause is not None:
            break
    if new_clause is None:
        return len(stack)
    stack.append(new_clause)

def minimal_order_of_modular_forms(clauses):
    # Placeholder implementation; actual computation depends on Hecke operators and lattices
    return len(clauses)

n = random.randint(5, 30)
d = random.randint(2, min(n - 1, 4))
formula = generate_tseitin_formula(n, d)
if formula is None:
    return {
        "metric_name": "resolution_length",
        "metric_value": float('inf'),
        "instances_tested": 0,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Graph size must be a multiple of the degree"
    }

resolution_len = resolution_length(formula)
modular_form_order = minimal_order_of_modular_forms(formula)

return {
    "metric_name": "resolution_length",
    "metric_value": resolution_len,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if "metric_value" in r) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if "metric_value" in r) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
''}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 15, 'instances_tested': 1, 'n_max': 15,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 30, 'instances_tested': 1, 'n_max': 30,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': inf, 'instances_tested': 0, 'n_max': 25,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': inf, 'instances_tested': 0, 'n_max': 21,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': inf, 'instances_tested': 0, 'n_max': 10,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': inf, 'instances_tested': 0, 'n_max': 23,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 12, 'instances_tested': 1, 'n_max': 12,
TRIAL: {'metric_name': 'resolution_length', 'metric_value': inf, 'instances_tested': 0, 'n_max': 11,

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67352eb4.py", line 95, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67352eb4.py", line 95, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data that could confirm or falsify the conjecture.
| next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17291
2	preregistration	ollama_remote	glm4:latest	0	0	19896
3	novelty	ollama_remote	glm4:latest	0	0	8219
4	novelty	ollama_remote	glm4:latest	0	0	9826
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29543
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13855
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9997
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9308

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	38073

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156008 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/16fe2475ef54.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/16fe2475ef54.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/16fe2475ef54.tar.gz (if generated)